CAPÍTULO 2. FUNDAMENTOS DE GIT

Introducción

Git es un sistema de control de versiones distribuido que permite registrar los cambios realizados en los archivos de un proyecto a lo largo del tiempo. Su principal función es conservar un historial de modificaciones, facilitar la recuperación de versiones anteriores y permitir que varias personas trabajen en un mismo proyecto sin perder el seguimiento de los cambios realizados.

En este capítulo se explican las operaciones fundamentales de Git, desde la creación de un repositorio hasta la administración de cambios, la consulta del historial y la conexión con repositorios remotos.

1. Obtener un repositorio Git

Un repositorio es el espacio donde Git almacena la información necesaria para controlar las versiones de un proyecto. Para comenzar a trabajar con Git existen dos opciones principales: crear un repositorio nuevo o descargar una copia de uno existente.

Para crear un repositorio nuevo se utiliza el comando:

```
git init
```

Este comando inicializa Git en la carpeta actual, permitiendo comenzar a registrar los cambios realizados en los archivos.

Por otra parte, cuando el repositorio ya existe y se encuentra alojado en una plataforma como GitHub, se puede descargar una copia mediante:

```
git clone URL DEL REPOSITORIO
```

El comando git clone descarga los archivos del proyecto y su historial de versiones, lo que permite trabajar localmente y posteriormente sincronizar los cambios.

2. Registrar cambios en el repositorio

Una de las principales características de Git es que permite seleccionar qué modificaciones se incluirán en cada registro del historial. El proceso habitual comprende tres etapas: modificar los archivos, preparar los cambios y crear una confirmación o commit.

Para conocer el estado actual del repositorio se utiliza:

```
git status
```

Este comando informa qué archivos han sido modificados, cuáles son nuevos y cuáles están preparados para ser confirmados.

Para seleccionar un archivo y preparar sus cambios se utiliza:

```
git add nombre_del_archivo
```

Después de preparar los cambios, se crea una confirmación mediante:

```
git commit -m "Descripción del cambio"
```

El comando git commit registra los cambios preparados en el historial del proyecto. El mensaje debe describir de manera clara y breve la modificación realizada.

Este procedimiento permite organizar el trabajo en unidades identificables y facilita la revisión de las modificaciones realizadas a lo largo del tiempo.

3. **Consultar el historial y las diferencias**

Git permite revisar los cambios efectuados en los archivos y consultar las confirmaciones anteriores. Estas funciones son importantes para verificar el trabajo antes de guardarlo definitivamente.

El comando:

```
git diff
```

muestra las diferencias entre los archivos modificados y la última versión registrada, teniendo en cuenta los cambios que todavía no se han preparado.

Para consultar las diferencias de los cambios que ya están preparados se utiliza:

```
git diff --staged
```

Por otra parte, el historial de confirmaciones puede consultarse mediante:

```
git log
```

Este comando muestra información de los commits, como su identificador, autor, fecha y mensaje. Para obtener una presentación más resumida se puede utilizar:

```
git log --oneline
```

Finalmente, el comando git show permite examinar los detalles de una confirmación específica, incluyendo los cambios asociados a ella.

4. Deshacer cambios y corregir errores

Git cuenta con herramientas que permiten corregir errores durante el desarrollo de un proyecto. Sin embargo, es importante identificar qué tipo de modificación se desea deshacer antes de ejecutar un comando.

Para retirar un archivo del área de preparación sin eliminar sus modificaciones locales se utiliza:

```
git restore --staged nombre_del_archivo
```

Si se desea descartar las modificaciones locales no confirmadas de un archivo, se puede utilizar:

```
git restore nombre_del_archivo
```

Este último comando debe emplearse con precaución, porque puede eliminar cambios que todavía no se han guardado en una confirmación.

Cuando el último commit contiene un mensaje incorrecto, es posible modificarlo mediante:

```
git commit --amend -m "Nuevo mensaje"
```

Esta operación permite corregir la confirmación más reciente. Es recomendable utilizarla con cuidado cuando el commit ya ha sido compartido con otras personas, debido a que modifica el historial.

En consecuencia, antes de deshacer o modificar cambios, es conveniente revisar el estado del repositorio y comprobar que no se perderá información importante.

5. Trabajar con repositorios remotos

Un repositorio remoto es una copia del proyecto alojada en otro equipo o servicio, como GitHub. Su función principal es facilitar la sincronización del trabajo y permitir la colaboración entre diferentes usuarios.

Para consultar los repositorios remotos configurados se utiliza:

```
git remote -v
```

Para descargar información de un repositorio remoto sin integrar automáticamente los cambios en la rama actual se utiliza:

```
git fetch origin
```

Para descargar e integrar cambios del remoto en la rama actual se utiliza:

```
git pull origin main
```

Por último, para publicar los commits locales en el repositorio remoto se utiliza:

```
git push origin main
```

En estos ejemplos, origin es el nombre habitual del repositorio remoto y main es el nombre de la rama principal. Ambos pueden variar según la configuración del proyecto.

La utilización de repositorios remotos permite mantener copias sincronizadas, respaldar el trabajo y facilitar la colaboración.

6. Etiquetas y alias

Las etiquetas (tags) permiten identificar puntos específicos del historial de un proyecto, especialmente aquellos relacionados con versiones importantes o entregas.

Para crear una etiqueta se utiliza:

```
git tag v1.0
```

Para consultar las etiquetas existentes se utiliza:

```
git tag
```

Por otra parte, Git permite crear alias, que son abreviaciones personalizadas para ejecutar comandos frecuentes.

Por ejemplo:

```
git config --global alias.st status
```

Este comando configura st como una abreviación de status. A partir de ese momento, se puede consultar el estado del repositorio mediante:

```
git st
```

Los alias ayudan a simplificar el trabajo cotidiano y permiten adaptar Git a las preferencias de cada usuario.

7. Ejercicio práctico

Para aplicar los conceptos estudiados, se propone crear un archivo, registrar sus cambios y consultar el historial del repositorio.

Primero, se crea un archivo de texto llamado notas.txt. Posteriormente, se revisa el estado del repositorio y se prepara el archivo para confirmar los cambios.

Los comandos que permiten desarrollar el ejercicio son:

```
echo "Mi primera práctica de Git" > notas.txt
git status
git add notas.txt
git commit -m "Agrega notas de práctica"
git log --oneline
```

Al ejecutar estos comandos, se crea el archivo, se prepara para su registro, se genera una confirmación y se consulta el historial. De esta manera, se practica el flujo básico de trabajo de Git.

Conclusión

Los fundamentos de Git permiten comprender cómo se administran las versiones de un proyecto y cómo se conserva un historial organizado de los cambios realizados. La creación de repositorios, el uso de comandos para preparar y confirmar modificaciones, la consulta de diferencias y la sincronización con repositorios remotos constituyen habilidades esenciales para trabajar de manera organizada.

Asimismo, es importante utilizar cuidadosamente los comandos que permiten deshacer cambios, con el fin de evitar la pérdida de información. El dominio de estas operaciones básicas proporciona las bases necesarias para estudiar temas más avanzados, como el trabajo con ramas, la resolución de conflictos y la colaboración mediante plataformas como GitHub.

Referencia bibliográfica

Chacon, S. y Straub, B. (2014). Pro Git (2.ª edición, versión en español). Capítulo 2: Fundamentos de Git. https://git-scm.com/book/es/v2